



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Estudio sobre el Kuzushiji-MNIST dataset

Laboratorio de Datos - Trabajo Práctico 02
2do. Cuatrimestre 2025
Departamento de Computación, FCEyN, UBA

Soriano Félix, Glionna Sofía, Gantman Ramiro

Nombre del grupo: Recursantes_FC

El Kuzushiji es el sistema de escritura japonés antiguo y obsoleto. En este trabajo se busca entrenar un modelo capaz de relacionar caracteres de dicho sistema con caracteres del japonés moderno, empleando el conjunto de datos Kuzushiji-MNIST. Se realizó una exploración inicial de los datos y se entrenaron diversos modelos de aprendizaje supervisado utilizando K-Nearest Neighbors y árboles de decisión, evaluados en distintos hiperparámetros y criterios de partición. En la clasificación binaria entre caracteres, el mejor modelo KNN alcanzó una exactitud del 98,55%, mientras que en la clasificación multiclase de los diez caracteres del dataset, el mejor árbol de decisión logró una exactitud del 71,05%.

Introducción

El Kuzushiji es el sistema de escritura japonés antiguo, previo a su modernización en el año 1868 donde se buscó simplificar el lenguaje y estandarizar la forma de los caracteres. En la actualidad los japoneses nativos no pueden leer kuzushiji sin entrenamiento especial. El proyecto “*Deep Learning for Classical Japanese Literature*”¹ busca entrenar inteligencia artificial para reconocer estos caracteres antiguos con el objetivo de que la población japonesa pueda acceder a fuentes históricas y literarias anteriores al siglo XIX. Crearon el Kuzushiji-MNIST dataset (entre otros, pero en particular este será de nuestro interés) que contiene datos de imágenes donde cada imagen del set de datos representa un carácter japonés antiguo manuscrito de entre 10 tipos distintos.

En el presente trabajo se estudió el Kuzushiji-MNIST. El objetivo será aplicar modelos de clasificación, como K-Nearest Neighbors (KNN) y árboles de decisión, para lograr predecir la clase correspondiente a un carácter a partir de sus píxeles. Para ello, se realizó un análisis exploratorio que permitió comprender la estructura y comportamiento de los datos, identificar atributos relevantes y evaluar su impacto en la clasificación. Posteriormente, se entrenó distintos modelos y comparó sus rendimientos bajo diversas configuraciones. Finalmente, se propone un modelo óptimo en base a los resultados obtenidos.

1 Análisis Exploratorio

El Kuzushiji-MNIST dataset cuenta con 70,000 filas y 785 columnas; 70,000 imágenes de 28x28 píxeles en escala de grises, donde cada clase cuenta con la misma cantidad de imágenes (7000 imágenes por cada clase). Cada fila representa un carácter japonés antiguo manuscrito, cada columna un pixel y los valores de la tabla indican el color de ese pixel para dicho carácter en una escala de grises de rango 0-255 donde “0” es negro (fondo) y “255” blanco (trazo). La última columna del dataset (columna 785) indica a qué clase de entre 10 pertenece el carácter antiguo en la modernidad.

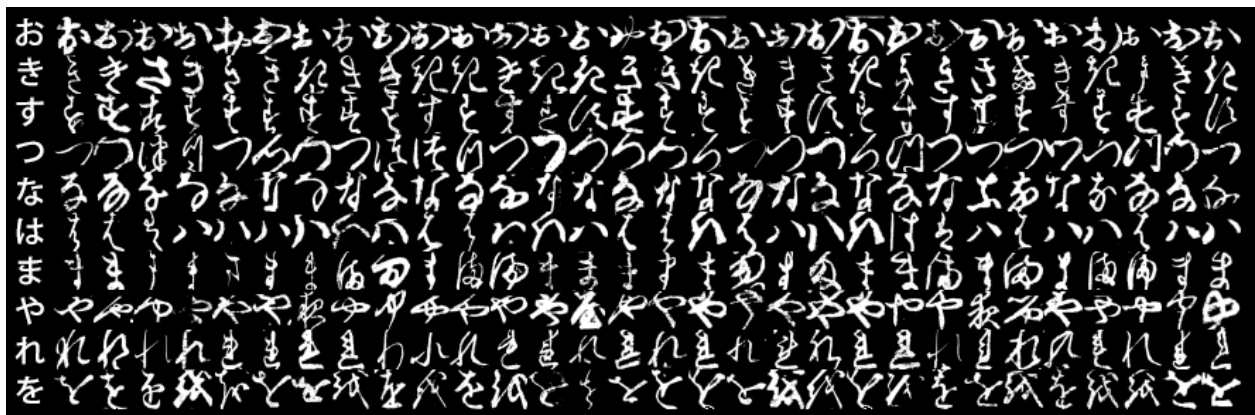


Figura 1. Las 10 clases de Kuzushiji-MNIST, con la primera columna mostrando el equivalente moderno.

¹ Clanuwat, Tarin, Mikel Bober-Irizar, Asanobu Kitamoto, Alex Lamb, Kazuaki Yamamoto, and David Ha. "Deep learning for classical japanese literature." arXiv preprint arXiv:1812.01718 (2018).

Al tratarse de un conjunto de datos compuesto por imágenes, el proceso de exploración y análisis inicial resulta más complejo que en aquellos casos donde los datos están representados mediante variables fácilmente interpretables (como números o categorías). Por ello, para su exploración fue necesario recurrir a técnicas de visualización, que permiten interpretar de manera más clara la información.

El estudio del dataset se ve dificultado debido a la presencia de caracteres muy distintos en una misma clase y caracteres muy similares a pesar de ser de clases distintas. Se puede observar en la figura 2 un ejemplo en el que se muestran dos caracteres pertenecientes a la clase 8 (れ) que difieren significativamente, inclusive pareciendo de distintas clases. A su vez, comparte similitudes con dos clases distintas; la clase 6 (ま) y la clase 1 (き).

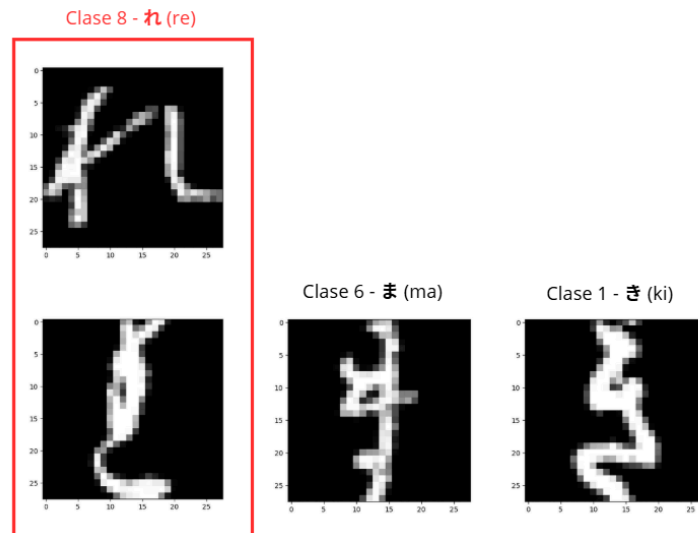


Figura 2. Similitudes entre distintos caracteres y diferencias dentro de una misma clase.

En el caso del caracter れ (Clase 8), existen hasta cuatro caracteres distintos a partir de los cuales puede escribirse, y algunos de ellos presentan incluso múltiples variaciones en su trazo. Este fenómeno no es exclusivo de dicho carácter: muchos símbolos presentes en el dataset Kuzushiji-MNIST poseen varias formas posibles de escritura. Esto incrementa la complejidad del problema de clasificación.

Con el propósito de analizar el comportamiento general de las clases, se realizó una visualización que muestra la superposición del valor promedio de cada píxel (Figura 3).

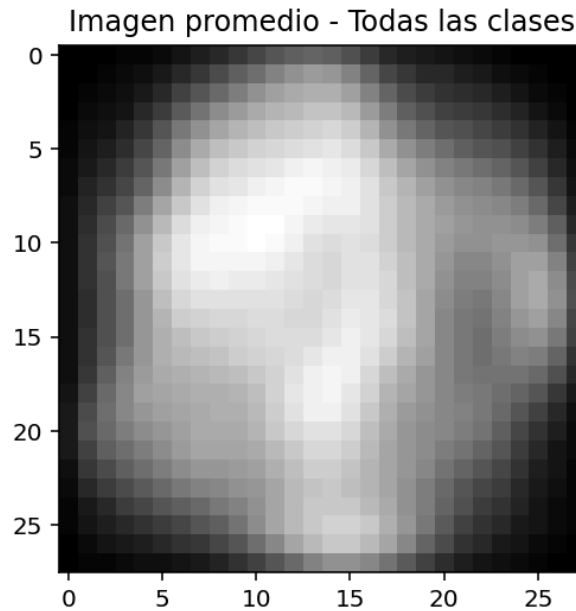


Figura 3. Promedio de pixeles coloreado en blanco (más saturación de píxeles) a negro (menos saturación de píxeles) entre todas las clases del dataset.

Se observó que los píxeles respetan cierto patrón dejando las esquinas de las imágenes en menor uso. Este comportamiento resultó útil para el posterior recorte de los datos, permitiendo identificar como zonas más relevantes aquellas con mayor saturación de píxeles y descartar las zonas menos saturadas.

2 Clasificación Binaria

Se buscó distinguir los caracteres de la clase 4 (な) y 5 (は) en particular. Para ello, primero se estudió la relevancia de sus píxeles teniendo en cuenta cómo difieren entre estas dos clases. Para poder clasificar atributos de más a menos relevantes, se analizaron los dataframes de las clases 4 y 5: Se tomó el promedio de cada atributo en cada clase, obteniendo (784×2) valores de rango 0-255. Este promedio indica cuán frecuente es que ese píxel se “pinte” en esa clase, a mayor promedio mayor frecuencia. Posteriormente, se restaron los promedios de ambas clases píxel a píxel. El valor absoluto de dicha resta representa, para cada píxel, la diferencia de cuanto se “pinta” en promedio entre ambas clases. Esta diferencia permite medir el grado de distinción de cada atributo: cuanto mayor es la diferencia, más relevante resulta dicho píxel para discriminar entre las clases 4 y 5, ya que revela una presencia marcada en una clase y escasa en la otra. Con este criterio se elaboró un ranking de atributos ordenados de mayor a menor relevancia.

Tomando los elementos de mayor relevancia se puede distinguir bien entre ambas clases. Pero es muy posible que 2 elementos cercanos en el dibujo tengan una relevancia similar por lo que podría resultar redundante tomar ambos.

Por esto se decidió experimentar tomando los píxeles más relevantes pero espaciados entre ellos. Es importante tener en cuenta que por más que los píxeles parezcan distantes en la fila de atributos, en la imagen estos pueden estar uno encima del otro. Por ello, diferencias numéricas entre atributos no siempre se traducen en diferencias visuales significativas. (Ver Figura 4).

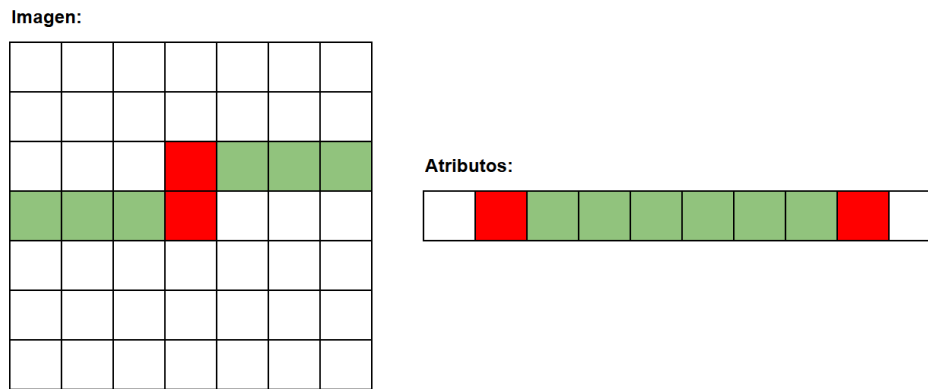


Figura 4. Representación gráfica de que dos atributos pueden parecer distantes en la lista (derecha) pero en la imagen son cercanos (izquierda).

Para esto se ideó un sistema que mida la distancia de Chebyshev entre 2 píxeles (Ver Figura 5) y se experimentó con diversas distancias mínimas permitidas entre los atributos seleccionados. En resumen se seleccionará el píxel de mayor relevancia siempre y cuando no esté a una distancia menor que la permitida (medidas usando la distancia de Chebyshev) de algún píxel ya seleccionado.

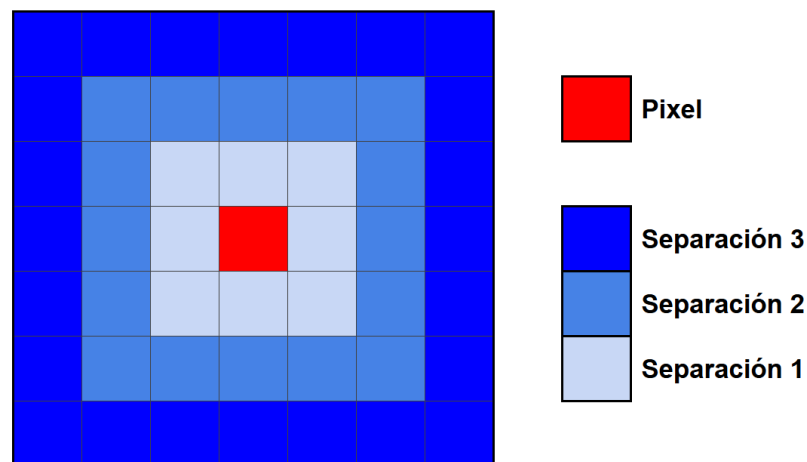


Figura 5. Representación gráfica de separación por píxel (distancia de Chebyshev).

Entonces, para una separación 3, se toman los primeros n elementos de la lista que cumplan esta condición. Las separaciones elegidas van desde 0 (simplemente se toman los primeros n elementos) hasta separación 3.

Independientemente del valor de k , para separaciones 2 y 3 el máximo de atributos que se pudo tomar con este sistema fue 71 y 41 respectivamente, por lo que en estos casos no fue posible tomar en cuenta cantidades superiores de atributos. Con los datos seleccionados, se entrenó el modelo KNN probando distintos hiperparámetros; un k de 3, 10 y 15; cantidad de atributos 3, 10, 20, 41, 50, 71 y 100; y separaciones de 0, 1, 2 y 3. Se presentaron los datos en una tabla para comparar aquellos modelos más exitosos. (Ver Tabla 1).

Top 10 modelos con mayor Accuracy

Valor de K	n atributos	Separación	Accuracy	Recall clase 4	Recall clase 5
3.0	100.0	1.0	0.985	0.978	0.992
10.0	100.0	1.0	0.983	0.977	0.99
15.0	100.0	1.0	0.981	0.969	0.993
3.0	71.0	2.0	0.978	0.964	0.991
10.0	71.0	2.0	0.978	0.968	0.988
15.0	71.0	2.0	0.975	0.96	0.99
3.0	50.0	2.0	0.975	0.962	0.988
3.0	71.0	1.0	0.975	0.96	0.99
10.0	50.0	2.0	0.971	0.957	0.987
10.0	71.0	1.0	0.97	0.956	0.985

Tabla 1. Los 10 modelos con mayor exactitud ordenados de forma descendente teniendo en cuenta todos los hiperparámetros.

Por lo observado en la tabla 1, se decide que el modelo óptimo entre los planteados es aquel que utiliza 100 atributos (de los seleccionados anteriormente), $k = 3$ y separación = 1 con un 98,55% de exactitud. Se puede observar que los modelos de igual cantidad de atributos mejoran a mayor separación (tener en cuenta que no se tomó casos de separación 3 con más de 41 atributos), además notar que aquellos modelos sin separación (separación = 0) ni siquiera aparecen en el top 10 de la tabla, teniendo el mejor de ellos una exactitud del 96%. Gracias a la separación, un modelo entrenado con 50 atributos resulta mejor que uno sin separación con 100 atributos, esto confirma que se toman píxeles redundantes cuando no se tiene separación.

3 Clasificación multiclase

En esta sección se busca entrenar un modelo de árbol de decisión que distinga entre las 10 clases del dataset. Con este fin se separaron los datos del Kuzushiji-MNIST; 80% para el desarrollo del modelo y un 20% para el held-out, que será utilizado finalmente para medir su exactitud.

Para aprovechar de la mejor forma la información disponible y reducir el tiempo de ejecución del programa, se buscó seleccionar los atributos que más información aportan en cuanto a la diferenciación de las clases. Para ello, se utilizó lo observado en el análisis exploratorio (Figura 3); Se utilizó el promedio de cada píxel por carácter (que ya había sido calculado) y se formó una lista que contiene todos los atributos del dataset ordenados de mayor a menor uso. Para evaluar la supuesta sospecha de que hay atributos redundantes se generaron tres modelos árboles de decisión utilizando el mismo criterio en todos (entropy) y se los entrenó con distintas

cantidades de atributos tomados de la lista de píxeles más importantes; 100 atributos, 350 atributos y 784 atributos (total). Para evaluar la performance de cada modelo, dentro del conjunto de desarrollo, se separó nuevamente un 20% para sus tests.

Los resultados se presentan a continuación (redondeados):

- Score del árbol 100 atributos con altura 10 = 60,82%
- Score del árbol 350 atributos con altura 10 = 70,40%
- Score del árbol 784 atributos con altura 10 = 71,76%

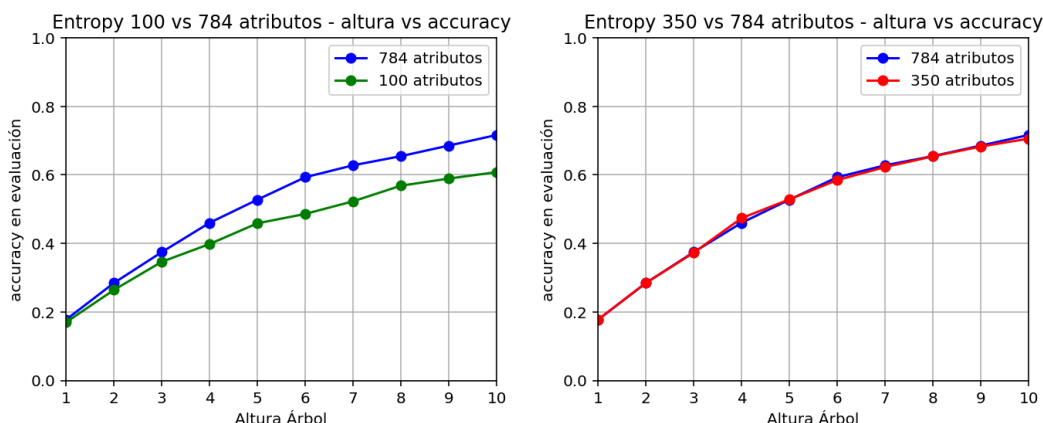


Figura 6. Modelo de árbol Entropy. Totalidad de los atributos comparada con muestras de atributos reducidas.

Se observó que el árbol con la totalidad de los atributos mejora tan solo en un 1,36% en comparación al árbol que toma 350 atributos, a pesar de estar tomando 434 atributos de menos. Esto comprueba lo que se pensaba; al tomar el total hay atributos redundantes. Sin embargo, el árbol que toma 100 atributos difiere considerablemente de aquel que toma el total, empeorando un 10,94%. Es por eso que se decidió que el mejor modelo es aquel que toma 350 atributos de la lista de píxeles más importantes ya que mejora notablemente el rendimiento sin sacrificar una exactitud significativa.

Teniendo en cuenta estos resultados, se entrenaron múltiples modelos de árbol de decisión sobre los 350 atributos seleccionados, variando sus hiperparámetros con el fin de identificar la configuración más adecuada. En particular, se modificó la profundidad máxima del árbol (1-10) y se utilizó tanto Gini como Entropy. Todos los entrenamientos fueron realizados aplicando un k-fold cross-validation con 5 particiones.

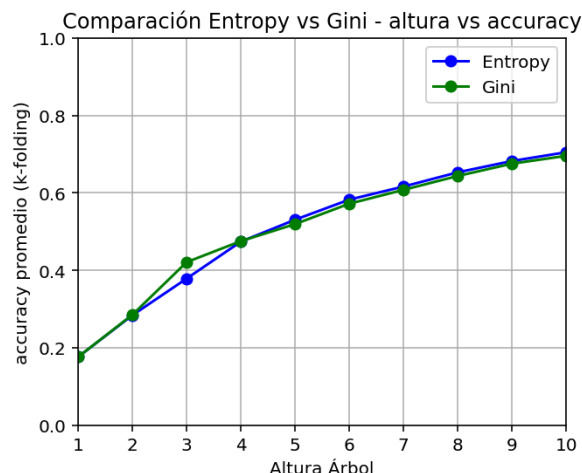


Figura 7. Comparación de exactitudes de Decision Tree (Gini y Entropy) según profundidad. Ambos modelos entrenados con k-fold cross-validation.

Como se observa en la figura 7, ambos modelos resultaron con una exactitud similar.

- Score del árbol Gini en altura 10 = 69,63%
- Score del árbol Entropy en altura 10 = 70,59%

Se tomó entonces como mejor modelo Entropy con 350 atributos tomados de la lista de píxeles más relevantes y una profundidad 10. Posteriormente se entrenó este modelo con todo el conjunto de desarrollo y se evaluó con el conjunto held-out separado al comienzo. Este modelo final resultó con una exactitud del 71,05%.

Matriz de Confusión - Held-Out (Entropy, profundidad=10)

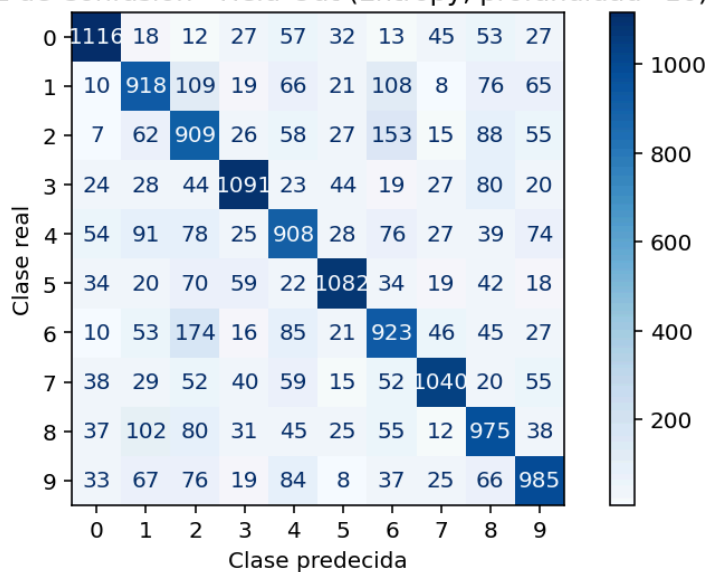


Figura 8. Matriz de confusión del modelo final evaluado con el conjunto held-out.

La matriz de confusión (Figura 8) permitió corroborar que la precisión obtenida no se debe a un reducido número de clases correctamente clasificadas, sino que evidencia un buen desempeño general en la identificación de los distintos caracteres. Además ayuda a ver que existen clases más similares que otras, como se había deducido anteriormente a simple vista. 174 atributos de la clase 6 fueron identificados como pertenecientes a la clase 2, mientras que solo 7 atributos de la clase 2 fueron clasificados como clase 0. Esto significa que el programa tiene mayor dificultad para distinguir ciertas clases de otras, seguramente debido a que estas son más semejantes en sus dibujos.

4 Conclusiones

En conclusión, en el Kuzushiji-MNIST dataset se pudieron encontrar patrones claros e información valiosa para el entrenamiento de modelos de clasificación, y se obtuvieron buenos resultados en cuanto al rendimiento de los mismos. El Análisis Exploratorio permitió identificar, mediante imágenes y análisis numéricos, que algunos atributos resultan más relevantes que otros para discernir entre clases.

En Clasificación binaria, se entrenó el mejor modelo KNN para clasificar aquellas clases pertenecientes a los caracteres な y は. Para ello, se guardó un 80% del total de las instancias para el entrenamiento de los modelos, y un 20% para testarlos; luego se clasificaron los píxeles más relevantes estratégicamente para distinguir estas dos clases en particular. Se realizaron pruebas comparando modelos con distintos hiperparámetros, como separación, y variando el valor de k, buscando la mejor exactitud posible. El mejor resultado obtenido en cuanto a la exactitud fue de 98,5%, con 100 atributos, $k = 3$ y 1 de separación. Siendo esta la separación más alta posible para esta cantidad de atributos, lo que demuestra que la estrategia de selección de atributos planteada resultó útil.

En Clasificación multiclase, se entrenó un modelo de árbol de decisión sobre un conjunto de desarrollo para distinguir entre las 10 clases del dataset. Gracias al análisis exploratorio de los datos se pudieron reconocer atributos que resultaban redundantes para así mejorar el rendimiento del modelo. El mejor modelo resultó ser Entropy con profundidad 10, obteniendo una exactitud del 71,05% evaluada sobre el conjunto held-out. Cabe aclarar que en el presente trabajo tan solo se probó con árboles de como máximo profundidad 10 y al medir el accuracy según profundidad la tendencia era ascendente (a mayor profundidad mayor accuracy). Suponemos que esta tendencia seguirá, por lo menos, con valores un poco más elevados. Por lo que se podrían lograr mejores resultados con árboles más profundos.

Bibliografía

- [1] Clanuwat, Tarin, Mikel Bober-Irizar, Asanobu Kitamoto, Alex Lamb, Kazuaki Yamamoto, and David Ha. ["Deep learning for classical japanese literature." \(2018\).](#)
- [2] ["Introduction to kuzushiji \(cursive Japanese calligraphy\)" \(PDF\).](#) National Institute for Japanese Language and Linguistics.
[Repositorio GitHub con análisis realizados.](#)